

Employee Management System Using HashSet and Layered Architecture

1. Case Study Overview

An organization wants a simple console-based Employee Management System to maintain employee records.

The application should allow users to:

- Add a new employee
- View all employees
- Search for an employee
- Update employee details
- Delete an employee
- Filter employees by department
- Sort employees by salary or name
- Calculate department-wise employee count
- Prevent duplicate employee records

The application will use a Java `HashSet` as the temporary in-memory data store.

No database is required.

2. Why Use HashSet?

A `HashSet` is suitable because:

- It does not allow duplicate elements.
- Employee lookup can be efficient.
- Addition and deletion are generally fast.
- It is useful when employee uniqueness must be maintained.
- It stores objects without maintaining insertion order.

The uniqueness of an employee will be determined using the employee ID.

For example:

- Employee ID: `EMP101`
- Employee ID: `EMP102`

Two employees with the same employee ID should be treated as duplicate employees, even when their names or departments are different.

3. Important HashSet Behaviour

A `HashSet` uses the following methods internally to identify duplicate objects:

- `hashCode()`
- `equals()`

Therefore, the `Employee` class must define employee equality based on the employee ID.

Example:

Employee 1:

- ID: EMP101
- Name: Rahul
- Department: IT

Employee 2:

- ID: EMP101
- Name: Priya
- Department: HR

Even though their names and departments are different, they must be considered the same employee because the employee IDs are identical.

4. Application Architecture

The project should follow layered architecture.

The application will contain the following layers:

1. Presentation Layer
2. Service Layer
3. Repository Layer
4. Model Layer
5. Exception Layer
6. Utility Layer

Application flow:

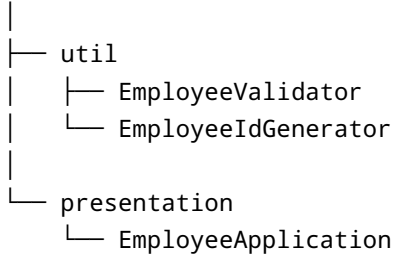
```
User
↓
Presentation Layer
↓
Service Layer
↓
Repository Layer
↓
HashSet<Employee>
```

The response travels back through the same layers:

```
HashSet<Employee>
↓
Repository Layer
↓
Service Layer
↓
Presentation Layer
↓
User
```

5. Suggested Package Structure

```
com.company.employee
|
├── model
│   └── Employee
|
├── repository
│   ├── EmployeeRepository
│   └── EmployeeRepositoryImpl
|
├── service
│   ├── EmployeeService
│   └── EmployeeServiceImpl
|
├── exception
│   ├── EmployeeNotFoundException
│   ├── DuplicateEmployeeException
│   └── InvalidEmployeeDataException
```



6. Employee Model Design

Create an Employee model class representing one employee.

Employee Attributes

The Employee class can contain the following fields:

Field	Data Type	Description
employeeId	String	Unique employee identifier
employeeName	String	Full name of employee
email	String	Employee email address
department	String	Employee department
designation	String	Employee job designation
salary	double	Monthly or annual salary
joiningDate	LocalDate	Date on which employee joined
employmentType	String	Permanent, Contract or Intern
active	boolean	Employee active status

Example Employee Record

```
Employee ID      : EMP101
Employee Name    : Abhishek Sharma
Email           : abhishek@company.com
Department      : Training
Designation     : Technical Trainer
Salary          : 85000
Joining Date    : 10 July 2026
```

Employment Type : Permanent
Active : Yes

7. Employee Identity Rule

The employee ID will act as the identity of an employee.

The `equals()` and `hashCode()` methods should use only the employee ID.

This means that:

EMP101, Rahul, IT
EMP101, Rahul Sharma, Training

will be treated as duplicate employees because both have the employee ID `EMP101`.

Do not use all employee fields in `equals()` and `hashCode()` because an employee's salary, designation, department or email may change.

8. Repository Layer

The repository layer is responsible for storing and retrieving Employee objects from the HashSet.

It should not contain business validation rules.

EmployeeRepository Interface

Define the operations that can be performed on employee data.

Recommended repository operations:

Method Responsibility	Description
Save employee	Add an employee to the HashSet
Find all employees	Return all employees
Find employee by ID	Search using employee ID
Delete employee by ID	Remove employee from HashSet

Method Responsibility	Description
Check employee existence	Verify whether ID is present
Update employee	Replace or modify an existing employee
Count employees	Return total number of employees
Find by department	Return department-specific employees

9. EmployeeRepositoryImpl Responsibilities

Create an implementation class for the repository interface.

This class will maintain:

HashSet<Employee>

as an instance-level collection.

Save Operation

The save operation should:

1. Receive an Employee object.
2. Add it to the HashSet.
3. Return whether the employee was added successfully.
4. Return false when a duplicate employee ID is detected.

Because HashSet does not allow duplicate elements, the `add()` operation will fail when an employee with the same employee ID already exists.

Find All Operation

The find-all operation should:

1. Access the employee HashSet.
2. Return all stored employees.
3. Avoid exposing the original collection directly where possible.
4. Return an empty collection when no employees are available.

Find by ID Operation

The repository should:

1. Receive an employee ID.
2. Iterate through the HashSet.
3. Compare the requested ID with each employee ID.
4. Return the matching employee.
5. Return an empty result when no employee is found.

An `Optional<Employee>` can be used to represent the result.

Delete Operation

The delete operation should:

1. Search for the employee using employee ID.
2. Remove the matching employee.
3. Return true when deletion succeeds.
4. Return false when the employee does not exist.

Update Operation

Updating an object inside a HashSet requires special care.

Do not directly change the employee ID of an employee already stored inside the HashSet.

Since employee ID participates in `equals()` and `hashCode()`, changing it while the object is inside the HashSet can corrupt collection behaviour.

The safe update process is:

1. Find the existing employee.
2. Remove the old employee object from the HashSet.
3. Update the permitted fields.
4. Add the modified employee back into the HashSet.

The employee ID should normally remain unchanged.

10. Service Layer

The service layer contains:

- Business validation

- Business rules
- Exception handling
- Employee-related calculations
- Coordination between presentation and repository layers

The service layer should not directly maintain the HashSet.

11. EmployeeService Interface

The service interface should describe the business operations available to the presentation layer.

Recommended operations:

- Register a new employee
 - Display all employees
 - Get employee by ID
 - Update employee details
 - Remove employee
 - Search employees by department
 - Search employees by designation
 - Search employees by joining year
 - Find employees with salary above a value
 - Sort employees by name
 - Sort employees by salary
 - Calculate total salary
 - Calculate average salary
 - Count employees by department
 - Activate or deactivate an employee
-

12. EmployeeServiceImpl Responsibilities

The implementation class should perform validation before calling the repository.

Employee Registration Process

The employee registration flow should be:

1. Receive employee details from the presentation layer.
2. Validate all employee fields.
3. Check whether the employee ID already exists.
4. Throw a duplicate employee exception if the ID exists.
5. Pass the valid employee to the repository.
6. Return a success message or status.

Find Employee Process

1. Receive the employee ID.
2. Validate that the ID is not blank.
3. Call the repository find-by-ID operation.
4. Throw `EmployeeNotFoundException` when the employee is unavailable.
5. Return the employee when found.

Delete Employee Process

1. Receive employee ID.
2. Validate the ID.
3. Check whether the employee exists.
4. Throw `EmployeeNotFoundException` if not found.
5. Call repository delete operation.
6. Return deletion status.

Update Employee Process

1. Receive employee ID and updated values.
 2. Validate the employee ID.
 3. Find the existing employee.
 4. Throw an exception when the employee does not exist.
 5. Validate the new values.
 6. Preserve the original employee ID.
 7. Remove the old employee record.
 8. Apply the updated fields.
 9. Add the employee back to the HashSet.
 10. Return the updated employee.
-

13. Employee Validation Rules

Create an `EmployeeValidator` utility class.

Employee ID Validation

- Must not be null.
- Must not be blank.
- Must follow a fixed format.
- Example format: `EMP101`.
- It should start with `EMP`.
- It should contain a numeric value after `EMP`.

Valid values:

EMP101
EMP102
EMP250

Invalid values:

101
EMP
employee101
EMP-101

Employee Name Validation

- Must not be null.
- Must not be blank.
- Should contain at least two characters.
- Should not contain numeric values.
- Leading and trailing spaces should be removed.

Email Validation

- Must not be null or blank.
- Must contain @.
- Must contain a valid domain section.
- It should preferably be unique across employees.

Department Validation

Allowed departments can be:

- Human Resources
- Finance
- Training
- Information Technology
- Sales
- Marketing
- Operations
- Administration

Invalid department values should be rejected.

Salary Validation

- Salary must be greater than zero.

- Salary should satisfy the minimum company salary rule.
- Example minimum salary: ₹15,000.
- Salary should not contain an invalid negative value.

Joining Date Validation

- Joining date must not be null.
- Joining date should not be an invalid future date unless future joining is allowed.
- Joining date can be stored using `LocalDate`.

Employment Type Validation

Allowed values:

- Permanent
- Contract
- Intern
- Consultant

14. Presentation Layer

The presentation layer communicates with the user.

It should:

- Display the menu.
- Accept user input.
- Convert input into the required data types.
- Call service-layer methods.
- Display success or error messages.
- Handle application flow.

The presentation layer should not directly interact with the HashSet.

15. Suggested Console Menu

```
=====
Employee Management System
=====

1. Add Employee
2. View All Employees
```

3. Search Employee by ID
4. Update Employee
5. Delete Employee
6. Search Employees by Department
7. Search Employees by Designation
8. Search Employees by Salary Range
9. Sort Employees by Name
10. Sort Employees by Salary
11. Display Department-wise Count
12. Display Salary Summary
13. Activate Employee
14. Deactivate Employee
15. Exit

Enter your choice:

16. Add Employee Use Case

Objective

Register a new employee in the system.

Input

- Employee ID
- Name
- Email
- Department
- Designation
- Salary
- Joining date
- Employment type

Processing Steps

1. User selects Add Employee.
2. Presentation layer collects employee details.
3. Employee object is created.
4. Employee object is sent to the service layer.
5. Service validates the employee.
6. Service checks duplicate employee ID.
7. Repository adds the employee to the HashSet.
8. Success message is displayed.

Success Message

Employee EMP101 added successfully.

Failure Conditions

- Duplicate employee ID
- Invalid email
- Blank employee name
- Invalid salary
- Invalid department
- Invalid joining date

17. View All Employees Use Case

Processing Steps

1. User selects View All Employees.
2. Presentation layer calls the service method.
3. Service calls the repository find-all method.
4. Repository returns the HashSet data.
5. Service returns employee records.
6. Presentation layer displays the employees.

Empty Collection Message

No employee records are available.

Display Format

```
Employee ID   : EMP101
Name          : Rahul Sharma
Department    : IT
Designation   : Software Engineer
Salary        : ₹75,000
Joining Date  : 10-07-2026
Status        : Active
-----
```

Because HashSet does not maintain order, employee records may appear in any order.

18. Search Employee by ID Use Case

Processing Steps

1. User enters an employee ID.
2. Service validates the ID.
3. Repository searches the HashSet.
4. Matching employee is returned.
5. Employee details are displayed.

Employee Not Found Message

Employee with ID EMP999 was not found.

19. Update Employee Use Case

Editable Fields

The following fields can be updated:

- Employee name
- Email
- Department
- Designation
- Salary
- Employment type
- Active status

The employee ID should not be changed.

Processing Steps

1. User enters employee ID.
2. Existing employee is searched.
3. Current employee information is displayed.
4. User enters new values.
5. New values are validated.
6. Existing employee is removed from the HashSet.
7. Updated values are applied.
8. Employee is added back to the HashSet.
9. Updated details are displayed.

20. Delete Employee Use Case

Processing Steps

1. User enters an employee ID.
2. Service validates the ID.
3. Existing employee is searched.
4. Employee is removed from the HashSet.
5. Deletion confirmation is displayed.

Success Message

Employee EMP101 deleted successfully.

Failure Message

Employee EMP101 does not exist.

21. Search Employees by Department

Processing Steps

1. User enters a department.
2. Service validates the department.
3. Repository returns all employees.
4. Service filters the employees by department.
5. Matching employees are returned.

Example

Input:

Department: Training

Output:

```
EMP101 - Abhishek - Technical Trainer
EMP108 - Priya - Senior Trainer
EMP115 - Rahul - Training Coordinator
```

22. Search Employees by Salary Range

The user should enter:

- Minimum salary
- Maximum salary

The service should:

1. Validate both salary values.
2. Ensure the minimum value is not greater than the maximum.
3. Retrieve all employees.
4. Filter employees whose salaries fall within the range.
5. Display the matching employee records.

Example:

```
Minimum Salary: 50000
Maximum Salary: 90000
```

23. Sorting Employees

HashSet does not maintain sorting.

Therefore, sorting should be performed by:

1. Retrieving employees from the HashSet.
2. Converting or copying them into a List.
3. Sorting the List using a Comparator.
4. Returning the sorted List to the presentation layer.

Sorting Options

Sort by Employee Name

- Alphabetical order

- Case-insensitive comparison

Sort by Salary

- Salary in ascending order
- Salary in descending order

Sort by Joining Date

- Oldest employee first
- Newest employee first

Sorting should not modify the original HashSet.

24. Salary Summary Operations

The service layer can provide the following calculations:

Total Salary

Calculate the total salary of all active employees.

Average Salary

Average Salary = Total Salary / Number of Employees

Highest-Paid Employee

Find the employee having the maximum salary.

Lowest-Paid Employee

Find the employee having the minimum salary.

Department-Wise Average Salary

Calculate the average salary separately for each department.

Example:

```
IT Department Average Salary      : ₹82,000
Training Department Average Salary : ₹74,500
HR Department Average Salary      : ₹65,000
```

25. Department-Wise Employee Count

The application should display how many employees work in each department.

Example output:

```
Information Technology : 12
Training               : 8
Human Resources        : 4
Finance                : 6
Sales                  : 10
```

This operation should be implemented in the service layer.

26. Employee Active and Inactive Status

Instead of deleting an employee who leaves the organization, the application can mark the employee as inactive.

Activate Employee

- Find employee by ID.
- Set active status to true.
- Save the updated employee.

Deactivate Employee

- Find employee by ID.
- Set active status to false.
- Save the updated employee.

This helps preserve employee history.

27. Exception Layer

Create custom exceptions for application-specific errors.

DuplicateEmployeeException

Raised when an employee ID already exists.

Example message:

```
Employee with ID EMP101 already exists.
```

EmployeeNotFoundException

Raised when an employee cannot be found.

Example message:

```
Employee with ID EMP999 was not found.
```

InvalidEmployeeDataException

Raised when employee data fails validation.

Example messages:

```
Employee name cannot be blank.  
Salary must be greater than zero.  
Joining date cannot be null.  
Invalid department value.
```

28. Exception Handling Flow

```
Repository Error  
  ↓  
Service Layer  
  ↓  
Custom Exception
```

↓
Presentation Layer
↓
User-Friendly Message

The presentation layer should catch the exceptions and display meaningful messages.

Avoid showing technical stack traces to the normal application user.

29. Employee ID Generation

Employee IDs can either be entered manually or generated automatically.

Automatic ID Format

```
EMP101  
EMP102  
EMP103
```

The EmployeeIdGenerator utility can maintain a sequence.

Processing:

1. Maintain the latest employee number.
2. Increment the number.
3. Prefix it with `EMP`.
4. Generate the next employee ID.

Example:

```
Current ID: EMP105  
Next ID: EMP106
```

For a simple in-memory application, the sequence will reset when the application restarts unless stored in a file or database.

30. Initial Sample Data

The application can load initial employee records when it starts.

Example records:

Employee ID	Name	Department	Designation	Salary
EMP101	Rahul Sharma	IT	Software Engineer	75000
EMP102	Priya Patil	Training	Technical Trainer	82000
EMP103	Neha Verma	HR	HR Executive	55000
EMP104	Amit Kumar	Finance	Accountant	62000
EMP105	Sneha Joshi	Sales	Sales Manager	90000

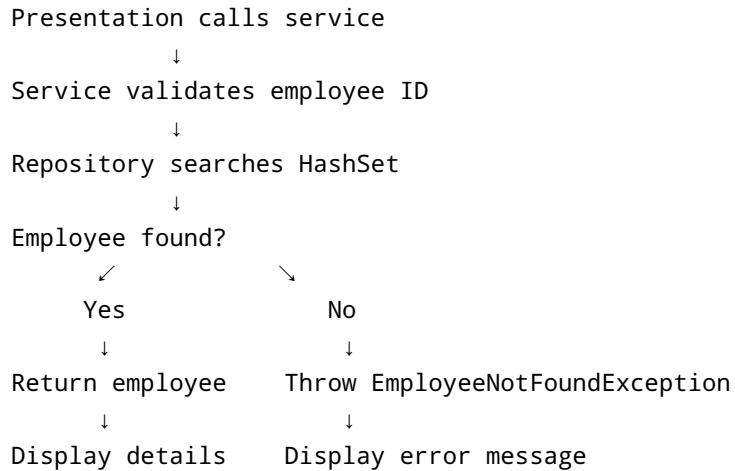
This data can be added to the HashSet during repository initialization.

31. Complete Add Employee Flow

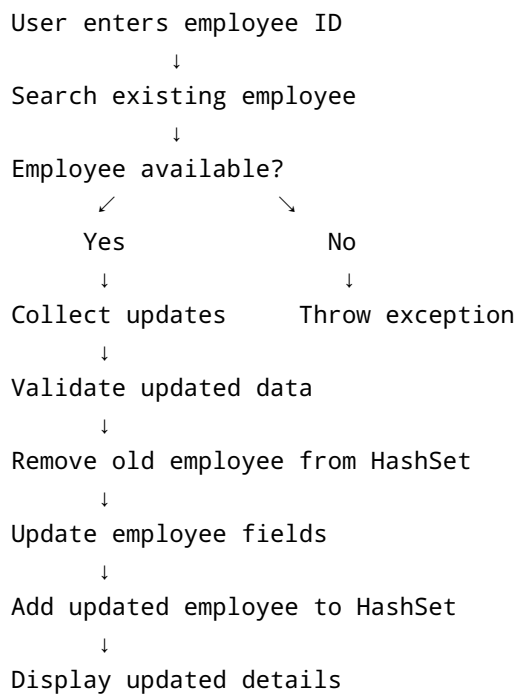
```
User selects Add Employee
↓
Presentation collects employee details
↓
Employee object is created
↓
EmployeeService.registerEmployee()
↓
EmployeeValidator validates fields
↓
Service checks duplicate employee ID
↓
EmployeeRepository.save()
↓
HashSet.add(employee)
↓
Success or duplicate result returned
↓
Message displayed to user
```

32. Complete Search Employee Flow

```
User enters employee ID
↓
```



33. Complete Update Employee Flow



34. Test Scenarios

Test Case 1: Add Valid Employee

Input:

```
Employee ID: EMP110  
Name: Kiran Patil  
Salary: 70000
```

Expected result:

```
Employee added successfully.
```

Test Case 2: Add Duplicate Employee

Input:

```
Employee ID: EMP110
```

Expected result:

```
Employee with ID EMP110 already exists.
```

Test Case 3: Invalid Salary

Input:

```
Salary: -5000
```

Expected result:

```
Salary must be greater than zero.
```

Test Case 4: Search Existing Employee

Input:

Employee ID: EMP101

Expected result:

Employee details should be displayed.

Test Case 5: Search Missing Employee

Input:

Employee ID: EMP999

Expected result:

Employee with ID EMP999 was not found.

Test Case 6: Delete Employee

Input:

Employee ID: EMP103

Expected result:

Employee deleted successfully.

Test Case 7: Filter by Department

Input:

Department: IT

Expected result:

All employees belonging to IT should be displayed.

Test Case 8: Update Salary

Input:

```
Employee ID: EMP102  
New Salary: 95000
```

Expected result:

Employee salary should be updated without changing the employee ID.

Test Case 9: Add Employee with Blank Name

Expected result:

```
Employee name cannot be blank.
```

Test Case 10: Sort Employees by Salary

Expected result:

Employees should be displayed in ascending or descending salary order.

35. Important Design Rules

1. Presentation layer must not access the HashSet directly.
 2. Repository layer should manage the HashSet.
 3. Service layer should contain validation and business rules.
 4. Employee ID should determine object uniqueness.
 5. `equals()` and `hashCode()` must be consistent.
 6. Do not change employee ID while the employee is inside the HashSet.
 7. Remove and re-add the employee during updates.
 8. HashSet does not guarantee insertion order.
 9. Use a List when sorted output is required.
 10. Use custom exceptions for business errors.
 11. Use `LocalDate` for joining date.
 12. Do not store scanner-related logic inside the service or repository layer.
-

36. Suggested Implementation Sequence

Phase 1: Create Project Structure

1. Create a Java project.
2. Create all required packages.
3. Verify package dependencies.

Phase 2: Create Employee Model

1. Define employee fields.
2. Add constructors.
3. Add getter and setter methods.
4. Define employee equality using employee ID.
5. Create a readable string representation.

Phase 3: Create Repository Layer

1. Create EmployeeRepository interface.
2. Define CRUD methods.
3. Create EmployeeRepositoryImpl.
4. Create the HashSet.
5. Implement save, find, update and delete operations.
6. Test duplicate prevention.

Phase 4: Create Exception Classes

1. Create DuplicateEmployeeException.
2. Create EmployeeNotFoundException.
3. Create InvalidEmployeeDataException.

Phase 5: Create Validation Utility

1. Validate employee ID.
2. Validate name.
3. Validate email.
4. Validate department.
5. Validate salary.
6. Validate joining date.
7. Validate employment type.

Phase 6: Create Service Layer

1. Create EmployeeService interface.
2. Create EmployeeServiceImpl.

3. Add validation.
4. Add duplicate checks.
5. Add business operations.
6. Add filtering and sorting.
7. Add salary calculations.

Phase 7: Create Presentation Layer

1. Create the main application class.
2. Display the menu.
3. Read user input.
4. Call service operations.
5. Handle exceptions.
6. Repeat the menu until Exit is selected.

Phase 8: Testing

1. Test valid additions.
2. Test duplicate employee IDs.
3. Test invalid values.
4. Test update and deletion.
5. Test filtering.
6. Test sorting.
7. Test salary calculations.
8. Test empty HashSet conditions.

37. Expected Learning Outcomes

After completing this case study, learners should understand:

- How HashSet stores custom objects
- How duplicate prevention works
- Importance of `equals()` and `hashCode()`
- How to design layered architecture
- Separation of concerns
- CRUD operations using collections
- Business validation
- Custom exception handling
- Filtering and sorting collection data
- Safe updating of HashSet elements
- Use of `LocalDate`
- Interface and implementation-based design